

Test-Time Training auto-supervisé pour la classification d'images

TER M1 VMI — état d'avancement

Maksym DOLHOV Fallou DIOUF

Université Paris Cité — M1 VMI

Mai 2026

Plan de l'exposé

- 1 Contexte et objectifs
- 2 Pipeline et méthode
- 3 Résultats préliminaires
- 4 Interprétation et prochaines étapes
- 5 Conclusion

C'est quoi une *distribution* ?

Réponse intuitive

Une distribution, c'est la **description de toutes les images que le modèle peut rencontrer** : lesquelles apparaissent, lesquelles sont fréquentes ou rares, et avec quelles étiquettes.

- On la note $\mathcal{D}$ et on écrit $x \sim \mathcal{D}$: x est tiré selon $\mathcal{D}$.
- On suppose les données d'entraînement **i.i.d.** (indépendantes, même loi) :

$$(x_1, y_1), \dots, (x_n, y_n) \sim \mathcal{D}_{XY}.$$

- Concrètement, $\mathcal{D}$ pour CIFAR-10 répond à : *quelles photos 32×32 peuvent apparaître, et avec quelles classes ?*

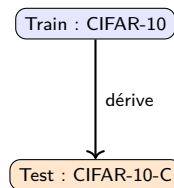
Et un *distribution shift* alors ?

Le modèle est entraîné sur $\mathcal{D}_{\text{train}}$, mais au test les données viennent d'une autre $\mathcal{D}_{\text{test}}$. **Les règles du jeu changent entre l'entraînement et le test.**

Dans notre cas : CIFAR-10 (images propres) $\rightarrow$ CIFAR-10-C (mêmes objets mais bruit, flou, météo).
Le *contenu sémantique* reste — un chien corrompu reste un chien — mais l'apparence change.

Contexte : robustesse à la dérive de distribution

- Un classifieur entraîné sur un jeu propre perd souvent en performance dès que la distribution change : bruit, flou, météo, artefacts numériques.
- Problème récurrent en déploiement réel.
- Deux leviers étudiés dans le TER :
 - **Pré-entraînement auto-supervisé** (SimCLR) pour des représentations plus robustes ;
 - **Test-Time Training** (Sun et al., 2020) pour adapter le modèle au moment du test.



corruptions × 5 sévérités

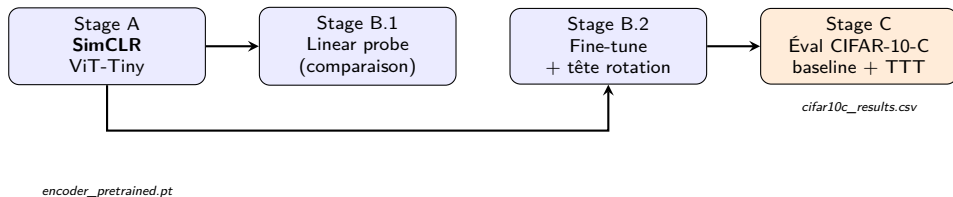
Objectifs du TER

- ① Implémenter un pipeline complet de bout en bout :
 - **SimCLR** pour le pré-entraînement auto-supervisé ;
 - **ViT-Tiny** comme encodeur ;
 - **CIFAR-10** pour l'entraînement, **CIFAR-10-C** pour l'évaluation sous dérive ;
 - **TTT** pour l'adaptation au temps du test.
- ② Mesurer l'effet de TTT (per-batch et per-image) sur l'accuracy.
- ③ Identifier les régimes (corruption, sévérité) où TTT apporte un gain réel.

Aujourd'hui

Le pipeline tourne de bout en bout, un premier run overnight a été réalisé sur GPU L4 ; nous présentons les résultats préliminaires et discutons des prochaines étapes.

Vue d'ensemble du pipeline



- Configuration centrale (`configs/default.yaml`), seeds fixes, AMP, AdamW + warmup/cosine.
- Préset *smoke* (5 min) et *default* (overnight L4).

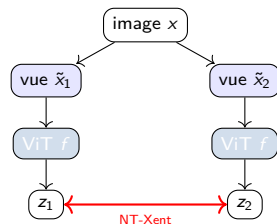
Stage A — SimCLR sur ViT-Tiny

Pré-entraînement auto-supervisé

- Deux vues augmentées par image (crop, color jitter, gray, blur).
- Perte contrastive **NT-Xent** sur la projection $z = g(f(x))$.
- Encodeur : ViT-Tiny, patch 4×4 , embed dim 192, drop_path 0.1.
- Projection MLP $\rightarrow$ 128-d.

Recipe

- 200 époques, batch 1024, $\tau = 0,2$, lr $3 \cdot 10^{-3}$, warmup 10 ép., cosine, AMP.



Stage B — Fine-tuning et tête auxiliaire

B.1 — Linear probe (comparaison)

- Encodeur gelé, classifieur linéaire, 30 époques. Sert de référence pour juger la qualité des représentations SimCLR.

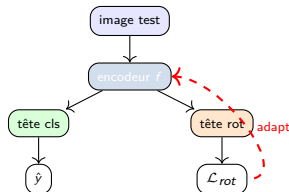
B.2 — Fine-tune supervisé (modèle principal)

- Encodeur dégelé, deux têtes en sortie :
 - tête **classification** (10 classes, perte cross-entropy + label smoothing 0.1) ;
 - tête **auxiliaire de rotation** (4 classes : 0° , 90° , 180° , 270°).
- Pertes combinées pendant l'entraînement.
- AdamW, lr $5 \cdot 10^{-4}$, weight decay 0.05, RandAugment, early stopping, AMP.
- La tête auxiliaire est conservée **pour TTT au moment du test**.

Stage C — Test-Time Training (Sun et al. 2020)

Idée

- Au test, on n'a pas les labels, mais on peut toujours résoudre la tâche auto-supervisée (rotation).
- Faire un pas de descente de gradient sur la perte rotation → adapter encodeur + tête rotation à la donnée de test.
- Puis prédire avec la tête de classification.



Deux régimes implémentés

- **per-batch** : un pas par batch de test ;
- **per-image** : un pas par image (128 copies tournées comme batch).

Paramètres TTT

lr=1e-3, steps=1, lambda_rot=1.0, adapt_scope=encoder_plus_head, snapshot/reset entre chaque (corruption, sévérité).

Module	Rôle
<code>src/core/</code>	pipeline orchestration, config dataclass
<code>src/data/</code>	datasets CIFAR-10 / CIFAR-10-C, 3 transforms
<code>src/models/</code>	ViT-Tiny backbone, SimCLR, classifier+rot head
<code>src/training/</code>	SimCLR / linear probe / fine-tune trainers
<code>src/ttt/</code>	adapter Sun 2020 (rotation), <code>rotate_batch</code>
<code>src/evaluation/</code>	boucle CIFAR-10-C avec/sans TTT, métriques
<code>src/utils/</code>	logger, checkpoints, seed
<code>configs/</code>	<code>smoke.yaml</code> , <code>default.yaml</code>
<code>notebooks/</code>	<code>colab.ipynb</code> (run de bout en bout)

- Code modulaire, ~2 500 lignes Python, configurable par YAML.
- Une seule entrée : `python main.py -config configs/default.yaml`.

- Run overnight unique sur GPU L4 (seed 42).
- **Stage C** : 14 corruptions¹ $\times$ 5 sévérités sur CIFAR-10-C, soit 70 cellules.
- Pour chaque cellule on calcule :
 - **baseline** (10 000 images, sans adaptation),
 - **per-batch TTT** (10 000 images, un pas par batch),
 - **per-image TTT** (sous-échantillon stratifié de 1 000 images, un pas par image).
- Per-image limité à 1 000 images : 128 copies tournées par image $\Rightarrow$ coût $\sim$ 20 h L4 sur les 70k.
- Reset des poids entre chaque (corruption, sévérité) pour éviter la fuite d'adaptation.

¹14 sur les 15 corruptions standard de Hendrycks 2019 — détail en **Annexe A**.

Accuracy sur données propres (CIFAR-10 clean)

Configuration	Top-1
Stage B.2 fine-tune (val, époque 30, best)	0,9008
Stage C clean — baseline (sans TTT)	0,8959
Stage C clean — per-batch TTT	0,8960 ($\Delta + 0,0001$)
Stage C clean — per-image TTT (sous-éch. 1k)	0,8980

- Aucune dégradation sur les données propres : la tête auxiliaire ne casse pas le classifieur quand on l'adapte au test.
- Bon sanity check avant de regarder CIFAR-10-C.

CIFAR-10-C — accuracy moyenne par sévérité

Sévérité	baseline	per-batch	per-image (1k)	Δ batch	Δ image*
1	0,8673	0,8679	0,8666	+0,0007	−0,0006
2	0,8392	0,8409	0,8395	+0,0017	+0,0003
3	0,8033	0,8059	0,8033	+0,0025	0,0000
4	0,7516	0,7575	0,7534	+0,0059	+0,0018
5	0,6803	0,6890	0,6830	+0,0087	+0,0027
moyenne	0,7883	0,7922	0,7892	+0,0039	+0,0009

- Per-batch : gain qui croît avec la sévérité (+0,07 pp à sév. 1 $\rightarrow$ +0,87 pp à sév. 5).
- * Δ image vs baseline 10k. Sur le sous-ensemble apparié 1k (apples-to-apples), per-image donne Δ moyen de −0,0048.

Wins / losses par cellule (70 cellules, sévérité ≥ 1)

Mode	wins	losses	ties
per-batch (vs baseline 10k)	61	6	3
per-image (vs baseline appariée 1k)	16	46	8

- **Per-batch** : 61/70 cellules améliorées, 6 régressions mineures, signature attendue de Sun 2020.
- **Per-image** : sur sous-ensemble apparié, 46/70 cellules régressent. Le mode dégrade nettement dans notre setup.

Top des gains per-batch (toutes sévérités confondues)

Corruption	sévérité	baseline	per-batch	Δ
fog	5	0,5367	0,5586	+0,0219
pixelate	5	0,7378	0,7530	+0,0152
impulse_noise	5	0,3959	0,4109	+0,0150
pixelate	4	0,7634	0,7772	+0,0138
impulse_noise	4	0,5547	0,5675	+0,0128
gaussian_noise	4	0,5556	0,5679	+0,0123

- Les gains se concentrent sur les corruptions *lourdes* (sév. 4–5) : brouillard, pixelisation forte, bruits.
- Sévérité 4 ressort souvent plus que la 5 : à sév. 5 le baseline est déjà très dégradé (gaussian_noise s5 = 49,6 %), il reste moins de signal à récupérer.
- Régimes où la tâche prétexte de rotation a le plus à donner.

Ablations TTT — effet de `steps` et `lr`

Trois configurations exécutées de bout en bout sur les 70 cellules de CIFAR-10-C :

Configuration	per-batch (Δ moyen)		per-image (Δ moyen, 1k matched)	
	toutes sév.	sév. 5 only	toutes sév.	sév. 5 only
Default (<code>steps=1</code> , <code>lr=1e-3</code>)	+0,39 pp	+0,87 pp	−0,48 pp	−1,24 pp
steps=5 (<code>lr=1e-3</code>)	+1,50 pp	+3,21 pp	−1,68 pp	−3,97 pp
lr=1e-4 (<code>steps=1</code>)	+0,04 pp	+0,10 pp	−0,05 pp	−0,11 pp

- **steps=5 amplifie les deux effets en miroir** : per-batch passe à 67/70 wins (max +6,78 pp à fog s5) ; per-image chute à 9/57 wins (min −9,20 pp à impulse_noise s5).
- **lr=1e-4 étouffe le gradient d'adaptation** $\Rightarrow$ TTT devient inopérant (signaux à ~ 0).
- Diagnostic : *plus on adapte, plus le per-image se dégrade* $\Rightarrow$ confirme l'hypothèse de sur-adaptation sur ViT/LN.

Résumé quantitatif des runs

Run	per-batch	per-image (1k)	wins/losses (per-batch)	wins/losses (per-image)
Default	+0,39 pp	−0,48 pp	61 / 6 / 3	16 / 46 / 8
steps=5	+1,50 pp	−1,68 pp	67 / 2 / 1	9 / 57 / 4
lr=1e-4	+0,04 pp	−0,05 pp	45 / 12 / 13	10 / 25 / 35

Format : wins / losses / ties sur 70 cellules (sévérité ≥ 1).

Lecture en une phrase

Seul **per-batch avec adaptation plus agressive (steps=5)** produit des gains francs et fiables (+1,5 pp moyen, +3,2 pp à sévérité 5, 67/70 cellules améliorées).

Per-image dégrade systématiquement, et l'effet s'aggrave quand on adapte plus.

- ❶ **Per-batch TTT fonctionne** comme attendu :
 - gain modeste mais cohérent qui croît avec la sévérité,
 - 61/70 cellules améliorées, aucun effet négatif sur les données propres,
 - compatible avec la signature publiée par Sun et al. 2020.
- ❷ **Per-image TTT dégrade** dans notre configuration ViT/LayerNorm :
 - ViT utilise **LayerNorm** : pas de régularisation par statistiques de batch, contrairement aux backbones BN sur lesquels TTT a été conçu ;
 - 128 copies tournées d'une seule image $\Rightarrow$ signal mince ;
 - le scope *encoder + head* peut être trop large pour un seul pas par image $\Rightarrow$ dérive de l'encodeur.
- ❸ **L'ablation steps=5 confirme l'hypothèse de sur-adaptation** : per-batch s'améliore encore (+1,5 pp moyen, 67/70 wins), per-image s'effondre davantage (−1,68 pp moyen, jusqu'à −9,2 pp). Adapter *plus* aide en mode batch et casse en mode image.
- ❹ **Gains concentrés sur les corruptions lourdes** : c'est là que le pretexte de rotation a le plus à apporter.

- Une seule seed $\Rightarrow$ pas de barres de variance.
- Per-image évalué sur 1 000 images seulement (contrainte de coût).
- Ablations partielles : seulement `steps` (1, 5) et `lr` ($1e-3$, $1e-4$) explorés ; `adapt_scope`, `lambda_rot` et `rotation_mode` pas encore balayés.
- Comparaison à TTT-BN (style Sun original avec ResNet) absente $\Rightarrow$ on ne peut pas isoler l'effet ViT/LN du reste.
- Pas encore de comparaison avec des baselines hors TTT (ex. Tent, BN-stats adaptation).

Prochaines étapes envisagées

- **Compléter les ablations** (déjà : steps=5, lr=1e-4) :
 - $\text{lr} \in \{10^{-4}, 10^{-3}, 10^{-2}\}$, steps $\in \{1, 5, 10\}$;
 - scope d'adaptation : *head only*, *LN only*, *encoder + head*.
- **Variance** : 3 seeds sur le pipeline complet pour des barres d'erreur.
- **Comparatif** : ResNet18 BN + TTT comme référence (vérifier l'hypothèse LN vs BN).
- **Aller plus loin** (selon le temps) : ActMAD, Tent, comparaison avec TTT-MAE.
- **Rédaction** du rapport final + figures consolidées (par corruption, courbes severité).

- Pipeline complet **opérationnel** : SimCLR (ViT-Tiny) → fine-tune avec tête rotation → évaluation CIFAR-10-C avec et sans TTT.
- Premier run overnight reproduit la signature attendue de TTT en mode **per-batch** (+0,4 pp en moyenne, croissant avec la sévérité).
- Mode **per-image** dégrade dans notre setup ViT/LN ⇒ piste d'investigation principale.
- Code public et reproductible :

`github.com/3v3r51nc3/SelfSupervised-TTT-ImageClassification`

Objectifs du rendez-vous

- 1 Vérifier que l'implémentation correspond aux attentes du sujet.
- 2 Discuter des prochaines ablations à prioriser.
- 3 Recueillir vos retours sur le mode per-image.

Questions pour vous — 1/2 : alignement et ablations

1. Alignement avec le sujet du TER

- ViT-Tiny (patch 4×4 , embed 192) — taille adaptée, ou faut-il viser ViT-Small ?
- Architecture en Y avec tête rotation entraînée *conjointement* en Stage B.2 — bien ce que vous attendiez, ou tête détachée préférable ?
- Choix de Sun 2020 (rotation) plutôt que TTT++ (NT-Xent) au moment du test — conforme au sujet, ou TTT++ attendu en plus ?
- Scope d'adaptation encoder + tête rotation (classifieur gelé) — bon choix, ou restreindre aux paramètres LN seulement ?
- Évaluation per-image sur 1 k images au lieu des 10 k — acceptable pour le rapport final ?

2. Ablations à prioriser

- 3 seeds pour variance — obligatoire, ou 1 seed acceptable au stade actuel ?
- ResNet18+BN comme expérience-contrôle pour isoler l'effet LN — indispensable, ou nice-to-have ?
- Ablation `adapt_scope` (`head_only` / `LN_only` / `encoder+head`) — prioritaire ?
- Ajout de `zoom_blur` pour passer à 15/15 corruptions — attendu pour le rapport ?
- Comparaison à au moins une baseline non-Sun (Tent ou ActMAD) — demandée ?

3. Ressources de calcul

- Notre crédit Colab Pro est épuisé. Possibilités côté laboratoire ou université :
 - GPU partagés du laboratoire LIPADE / accès SSH ?
 - Budget Google Cloud / AWS éducation ?
- Estimation de notre besoin restant : $\sim 30\text{--}40$ h L4 équivalent ($3 \text{ seeds} \times 2 \text{ configs} + \text{ResNet contrôle} + \text{ablations adapt_scope}$).

4. Livrables et organisation

- Format et longueur attendus pour le rapport final ?
- Date approximative de la soutenance ?

Références I

- [1] Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, M. Hardt.
Test-time training with self-supervision for generalization under distribution shifts.
ICML, 2020. [arXiv:1909.13231](#).
- [2] Y. Liu, P. Kothari, B. Van Delft, B. Bellot-Gurlet, T. Mordan, A. Alahi.
TTT++: When does self-supervised test-time training fail or thrive?
NeurIPS, 2021.
- [3] J. Han, J. Park, D. Han, W. Hwang.
When Test-Time Adaptation Meets Self-Supervised Models.
[arXiv:2506.23529](#), 2025.
- [4] M. J. Mirza, P. Jané Soneira, W. Lin, M. Kozinski, H. Possegger, H. Bischof.
ActMAD: Activation matching to align distributions for test-time training.
CVPR, 2023.
- [5] T. Chen, S. Kornblith, M. Norouzi, G. Hinton.
A Simple Framework for Contrastive Learning of Visual Representations (SimCLR).
ICML, 2020. [arXiv:2002.05709](#).

- [6] A. Dosovitskiy et al.
An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT).
ICLR, 2021. [arXiv:2010.11929](#).
- [7] D. Hendrycks, T. Dietterich.
Benchmarking Neural Network Robustness to Common Corruptions and Perturbations (CIFAR-10-C).
ICLR, 2019. [arXiv:1903.12261](#).
- [8] S. Gidaris, P. Singh, N. Komodakis.
Unsupervised Representation Learning by Predicting Image Rotations.
ICLR, 2018. [arXiv:1803.07728](#).
- [9] A. Krizhevsky.
Learning Multiple Layers of Features from Tiny Images (CIFAR-10/100).
Tech. report, U. Toronto, 2009.

Merci de votre attention

Questions ?

Maksym DOLHOV ■ Fallou DIOUF

TER M1 VMI — Université Paris Cité — Mai 2026

Annexes

Slides de réserve — Q&A préparé

Annexe A — Pourquoi 14 corruptions et non 15 ?

Q : « *Le benchmark CIFAR-10-C standard (Hendrycks & Dietterich, 2019) en compte 15. Pourquoi 14 ?* »

- Notre liste codée en dur (`src/data/dataset.py`) couvre 14 des 15 corruptions standard ; il manque `zoom_blur`.
- Repéré après le run overnight en relisant Hendrycks — *oubli* de notre part dans la liste, pas un choix méthodologique.
- Les 4 **familles** du benchmark (noise, blur, weather, digital) restent toutes représentées :
 - noise : gaussian, shot, impulse blur : defocus, glass, motion
 - weather : snow, frost, fog, brightness digital : contrast, elastic, pixelate, jpeg
- La lecture qualitative (*per-batch fonctionne, per-image dégrade, gains croissants avec sévérité*) ne dépend pas d'`zoom_blur`.
- **Action** : ajout d'`zoom_blur` et rerun de Stage C avant le rapport final (~30 min de marginal sur L4).

Annexe B — Seed unique, comment quantifier le bruit ?

Q : « Comment savez-vous que le +0,4 pp de per-batch n'est pas du bruit de seed ? »

- **Honnêtement : nous ne le savons pas encore.** Le run présenté est sur une seule seed (42).
- Indices indirects qui suggèrent que ce n'est pas du bruit pur :
 - monotonie sur la sévérité (+0,07 $\rightarrow$ +0,87 pp), peu probable par chance ;
 - 61/70 cellules améliorées (binomial $p < 10^{-12}$ contre 50/50) ;
 - zéro régression sur clean — adaptation contrôlée.
- Plan : 3 seeds (42, 1337, 2024) avec mêmes hyperparamètres pour barres d'erreur ; coût ~ 6 h L4.
- Pour la per-image dégradation ($\Delta = -0,48$ pp sur 1k), le signe est cohérent à travers toutes les sévérités ≥ 1 ; même argument de monotonie tient.

Annexe C — ViT/LN vs ResNet/BN : choix défendable ?

Q : « Sun 2020 utilise ResNet-26 + BatchNorm. Vous changez backbone ET norme : comment isoler les effets ? »

- Choix de ViT-Tiny motivé par :
 - le sujet du TER demande explicitement *ViT comme encodeur* (cf. consigne TER2.pdf) ;
 - compatibilité directe avec SimCLR (representations [CLS]) sans modification d'architecture ;
 - coût gérable sur L4 vs ViT-S/B.
- Conséquence reconnue : LayerNorm n'a pas de statistiques de batch à recalibrer $\Rightarrow$ TTT par batch *ne peut pas* bénéficier du même mécanisme que le TTT-BN original.
- Notre +0,4 pp est donc **purement par adaptation des poids via la perte rotation**, pas via une renormalisation BN.
- **Ablation prévue** : refaire l'expérience avec ResNet-18 + BN dans le même pipeline pour isoler la contribution de la norme.

Annexe D — L'hypothèse « per-image dégrade à cause de LN » est-elle testée ?

Q : « Vous avancez 3 hypothèses pour la régression per-image. Laquelle est démontrée ? »

- **Aucune n'est démontrée**, ce sont des hypothèses ordonnées par plausibilité — nous l'assumons explicitement.
- Plan d'ablation pour *falsifier* chacune :
 - **LN vs BN** : ResNet-18 + BN identique. Si per-image gagne sur ResNet $\Rightarrow$ hypothèse 1 confirmée.
 - **Signal mince (128 copies)** : varier `per_image_aug_k` $\in \{16, 64, 128, 256\}$. Si la dégradation s'atténue avec k croissant $\Rightarrow$ hypothèse 2.
 - **Scope trop large** : `adapt_scope` $\in \{\text{head_only}, \text{ln_only}, \text{encoder+head}\}$. Si *ln_only* ou *head_only* récupère, hypothèse 3.
- Runs `ablation-lr-1e-4` et `ablation-steps-5` déjà lancés ; structure prête pour les autres axes.

Q : « Sur quoi avez-vous tuné lr , $steps$, $lambda_rot$, $adapt_scope$? Pas sur CIFAR-10-C j'espère ? »

- Hyperparamètres TTT **n'ont pas été tunés** sur CIFAR-10-C : valeurs reprises directement de Sun 2020 ($lr=1e-3$, $steps=1$, $lambda_rot=1.0$).
- Hyperparamètres SimCLR / fine-tune choisis sur la **val split** de CIFAR-10 (10% du train), CIFAR-10-C jamais vu pendant le tuning.
- Les ablations à venir (lr , $steps$, $scope$) seront menées sur le **validation set des 4 corruptions « extra »** de Hendrycks ($speckle_noise$, $gaussian_blur$, $spatter$, $saturate$), précisément le rôle pour lequel elles sont conçues.
- Le rapport final reportera les nombres finaux uniquement sur les 15 corruptions « test », sans tuning post-hoc.

Annexe F — Pourquoi pas de comparaison avec Tent / SAR / BN-stats ?

Q : « *Sun 2020 c'est 2020. Pourquoi pas Tent (2021), EATA (2022), SAR (2023) ?* »

- Le sujet du TER cite explicitement **Sun 2020** [1] et **TTT++ (Liu 2021)** [2] comme références principales — nous avons commencé par [1] qui est la formulation canonique du TTT auto-supervisé.
- Comparer à Tent / SAR sortirait du cadre direct du sujet et nécessiterait :
 - d'implémenter une seconde tête / mécanisme d'adaptation ;
 - de standardiser les budgets de calcul (Tent fait 1 backward sur entropy seulement, Sun fait un pas via rotation, ce n'est pas directement comparable).
- Extension naturelle si le temps le permet : ajouter Tent ou TTT++ comme seconde baseline pour situer notre résultat dans le paysage TTT.
- Tent / EATA / SAR mentionnés en related work du rapport.

Annexe G — Coût computationnel de TTT

Q : « Vous gagnez 0,4 pp avec per-batch et 4–8 pp à sévérité 5. Combien ça coûte en temps d'inférence ? »

Mode	gain moyen	sur-coût d'inférence
baseline	—	1× (forward seul)
per-batch TTT	+0,4 pp ($\sim +0,9$ pp à sév. 5)	$\sim 2\times$ (1 fwd + 1 bwd)
per-image TTT	−0,5 pp	$\sim 130\times$ (128 copies + bwd)

- Per-batch double l'inférence pour un gain modeste — compromis raisonnable en déploiement où la robustesse compte.
- Per-image **n'est jamais rentable** dans notre setup : plus lent ET pire en accuracy.
- Cost-accuracy curve à inclure dans le rapport.

Annexe H — Reset entre (corruption, sévérité) : choix réaliste ?

Q : « *En vrai déploiement on a un stream continu, pas 70 cases isolées. Pourquoi reset entre chaque ?* »

- Reset est un choix **méthodologique** pour mesurer *l'effet de TTT en isolation*, conformément au protocole de Sun 2020 :
 - sans reset, l'adaptation cumule et masque la contribution propre à chaque type de dérive ;
 - avec reset, on lit chaque (corruption, sévérité) comme un test indépendant.
- Pas réaliste pour un déploiement, oui — mais c'est l'angle du papier de référence.
- Le scénario *continual TTT* (sans reset, dérive qui change dans le temps) est une ligne de recherche distincte (Wang et al. 2022, EATA) qui pourrait constituer une extension du TER.
- Notre code expose le reset comme un toggle $\Rightarrow$ trivial à désactiver pour une expérience continual.

Annexe I — Pourquoi SimCLR et rotation, pas MAE / DINO ?

Q : « 2020 c'est ancien. SimCLR + rotation, pas un peu daté pour un ViT ? »

- Choix dicté par le sujet TER (SimCLR pour pré-entraînement, Sun 2020 pour TTT).
- Justification scientifique néanmoins :
 - SimCLR reste un baseline contrastif solide, bien compris, peu coûteux (~ 2 h L4 vs ~ 10 h pour MAE comparable) ;
 - Rotation est la tâche prétexte la plus simple à coupler avec une tête auxiliaire pendant le fine-tune ;
 - Plus important : on étudie *l'effet de TTT*, pas l'optimum de SSL. Tant que la représentation est compétente, la conclusion sur TTT reste valide.
- Extension possible : remplacer SimCLR par DINO ou MAE et regarder si le signal TTT change qualitativement.

Annexe J — « Où est SimCLR exactement ? »

Q : « Vous deviez utiliser SimCLR. Or au moment du test vous adaptez via une perte rotation. SimCLR est-il vraiment utilisé ? »

Oui, SimCLR est central dans le pipeline — mais à une étape précise :

Étape	Tâche prétexte	Loss
Stage A — pré-entraînement	SimCLR (2 vues augmentées)	NT-Xent , 200 époques
Stage B.2 — fine-tune	rotation auxiliaire (4 angles)	CE classif + CE rotation
Stage C — TTT (Sun 2020 [1])	rotation auxiliaire	CE rotation seule

- Les représentations de l'encodeur ViT sont **intégralement issues de SimCLR** (Stage A) — sans cette pré-tâche, pas de modèle.
- Au moment du *test*, le sujet [1, Sun 2020] spécifie la rotation comme tâche prétexte d'adaptation $\Rightarrow$ c'est ce que nous utilisons.
- La projection SimCLR est jetée après Stage A (pratique standard du papier original).

Annexe K — Et si vous vouliez du contrastive *aussi* pendant le TTT ?

Q : « *Pourquoi ne pas utiliser NT-Xent au moment du test, comme TTT++ (Liu 2021) ?* »

- C'est la variante de la deuxième référence du sujet : **[2] Liu et al., TTT++ (NeurIPS 2021)**. Conceptuellement légitime, complémentaire de Sun 2020 [1].
- **Notre choix actuel** : commencer par [1] Sun 2020 (rotation), formulation canonique et plus simple à implémenter de bout en bout.
- **Extension TTT++ peu coûteuse à ajouter** : notre pipeline est structuré pour, il suffirait de :
 - conserver la tête de projection SimCLR au-delà de Stage A (au lieu de la jeter),
 - à l'inférence : générer 2 vues augmentées par image, calculer NT-Xent, faire un pas SGD,
 - remplacer `_rotation_step()` par `_contrastive_step()` dans `src/ttt/adapter.py` (~50 lignes).
- **Proposition** : si vous le jugez prioritaire, nous pouvons ajouter TTT++ comme seconde variante TTT et comparer rotation vs NT-Xent au test.

Annexe L — Pourquoi notre question sur la tête rotation conjointe ?

Q : « Architecture en Y avec tête rotation entraînée conjointement en Stage B.2 — bien ce que vous attendiez, ou tête détachée préférable ? »

Notre setup actuel (`src/training/ttt_finetune_trainer.py`)

- Stage B.2 : encodeur + tête classification + tête rotation entraînés *ensemble*.
- Loss combinée : $\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \cdot \mathcal{L}_{\text{rot}}$, avec $\lambda = 1,0$.

D'où vient la question

- $\lambda = 1$ vient de Sun 2020, mais sur **ResNet + BN**. Sur **ViT + LN**, l'équilibre peut être différent.
- Avec entraînement conjoint, la perte rotation *tire* l'encodeur dans sa direction. Si λ trop grand $\rightarrow$ classification souffre. Trop petit $\rightarrow$ tête rotation mal calibrée et signal TTT faible en Stage C.
- Notre gain per-batch modeste en default (+0,39 pp) pourrait s'expliquer en partie par un encodeur *tirailé* entre les deux objectifs.

Alternative : tête détachée — entraîner d'abord le classifieur seul, puis figer l'encodeur et entraîner la tête rotation séparément. Conséquence : encodeur plus précis pour la classif, mais moins adapté à la tâche prétexte $\Rightarrow$ signal TTT en Stage C plus faible.

Pourquoi on demande : valider le schéma avant de tuner λ ou de tester l'alternative.

Annexe M — Pourquoi notre question sur `adapt_scope` ?

Q : « Scope d'adaptation encoder + tête rotation — bon choix, ou restreindre aux paramètres LN seulement ? »

Setup actuel (`adapt_scope = encoder_plus_head`) : un pas SGD met à jour l'encodeur ViT-Tiny entier ($\sim 5,7$ M params) + tête rotation (~ 770 params), classifieur gelé. Soit $\sim 99\%$ du modèle par gradient.

Alternative `ln_only` : ne mettre à jour que les paramètres affines LayerNorm (γ, β) , ~ 9 k params, soit $\sim 0,15\%$ du modèle.

D'où vient la question

- TENT (Wang 2021) montre qu'adapter uniquement les paramètres affines de la norme est souvent plus stable que d'adapter l'encodeur entier.
- Notre per-image dégrade (annexe D, hypothèse 3 : scope trop large $\rightarrow$ dérive sur gradient mince). `ln_only` contraindrait l'adaptation.
- Restreindre aux LN reste conceptuellement dans Sun 2020 (perte rotation, juste moins de degrés de liberté).

Pourquoi on demande : valider que cette ablation est une extension légitime de Sun 2020 sous ViT/LN, et non un glissement vers TENT.